

```

my_probe.kp.addr = (kprobe_opcode_t *)0xc071c9a9;
my_probe.entry = (kprobe_opcode_t *)my_handler;
register_jprobe(&my_probe);
return 0;
}

void myexit(void)
{
    unregister_jprobe(&my_probe);
    printf("module removed\n");
}

module_init(myinit);
module_exit(myexit);

/*Kernel module Comments*/
MODULE_AUTHOR("Manoj");
MODULE_DESCRIPTION("SIMPLE MODULE");
MODULE_LICENSE("GPL");
/*MODULE_LICENSE("GPL v2");

```

## Code walk-through

The example is simple to understand, but let me explain things a bit. Here, in the `myinit()` function, you assigned the target function address to the `addr` member of the Kprobe member struct `'kp'`, just like for the earlier module. The main difference is that you've now assigned a single handler function, `my_handler`, to the `entry` member:

```
my_probe.entry = (kprobe_opcode_t *)my_handler;
```

You've probably already noted that the signature of the single handler function here is quite different from the Kprobe handlers. The reason is, the handler must have the same arguments as that of the kernel function you're probing, which is once again `ip_rcv()`:

```

int my_handler (struct sk_buff *skb, struct net_device *dev,
struct packet_type *pt, struct net_device *orig_dev);
extern int ip_rcv(struct sk_buff *skb, struct net_device *dev,
struct packet_type *pt, struct net_device *orig_dev);

```

Jprobe lets us access the arguments of a function by calling your handler with the same arguments passed to the target function. This means that when `ip_rcv` is called, its arguments can be accessed from your probe handler as it is able to refer to the function's address space plus the components within that function stack.

The line `my_iph = ip_hdr(skb)`; will extract the IP header from `sk_buff`. Then extract the source and destination IP addresses in dot notation form, using the `NIPQUAD` and `NIPQUAD_FMT` macros declared in `include/linux/kernel.h`, and print the addresses.

Now, compile your module, insert it, and check that the module has been inserted successfully, just as before. Again, to invoke `ip_rcv()`, run a `ping` and then run `dmesg` to check the output:

```

# ping www.google.com
# dmesg
Source IP: 192.168.1.1
Destination IP: 192.168.1.3
Source IP: 299.85.231.194
Destination IP: 192.168.1.3

```

The output shows that Jprobe lets you get the function's argument values, which can be very handy when debugging data-dependent bugs. 

## By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux Kernel Programming, Linux Administration, Cluster Computing, Embedded Systems and QT/GTK programming on Linux. You can reach him at [manoj@nixitsolutions.com](mailto:manoj@nixitsolutions.com) and read his blog, [www.open4india.blogspot.com](http://www.open4india.blogspot.com), to share your Linux knowledge.

## Go Online to learn Linux Device Drivers



Core skills like kernel programming or Linux device drivers should be learned at a slow and steady pace for fully comprehending it, says Mr. Raghu Bharadwaj, a lead trainer on core Linux programming. Unlike normal application software learning where it is mostly restricted to functionality based learning, system programming involves deep understanding and analysis of issues like the Linux kernel and its subsystems. Your options to learn these skills are classroom based training, learning through experience, self learning through trial and error and through an effective online course.

While all other modes of training has its own merits and demerits, online learning is definitely a step ahead, especially for courses like this. Online courses gives you the comfort of accessing it 24x7 and lets you learn things during hours of focus and interest. Another important aspect is that it permits you to go through each session any number of times and fully understand each and every topic effectively.

Mr. Raghu Bharadwaj is a lead trainer at Veda Solutions, you can listen to his **training videos** on Linux kernel and device drivers at [www.techveda.org](http://www.techveda.org)

**First Online Course on Linux Device Drivers in India**  
[www.techveda.org](http://www.techveda.org)